



数据结构与算法分析-4



人工智能与自动化学院

韩守东

shoudonghan@hust.edu.cn

13720301586



上节课回顾

▶ 两种受限的线性结构

- 栈：先进后出,在表尾操作
- 队列：后进先出，队首删除，队尾插入

▶ 基本操作

- ▶ 栈：Push, Pop, GetTop
- ▶ 队列：EnQueue, DeQueue

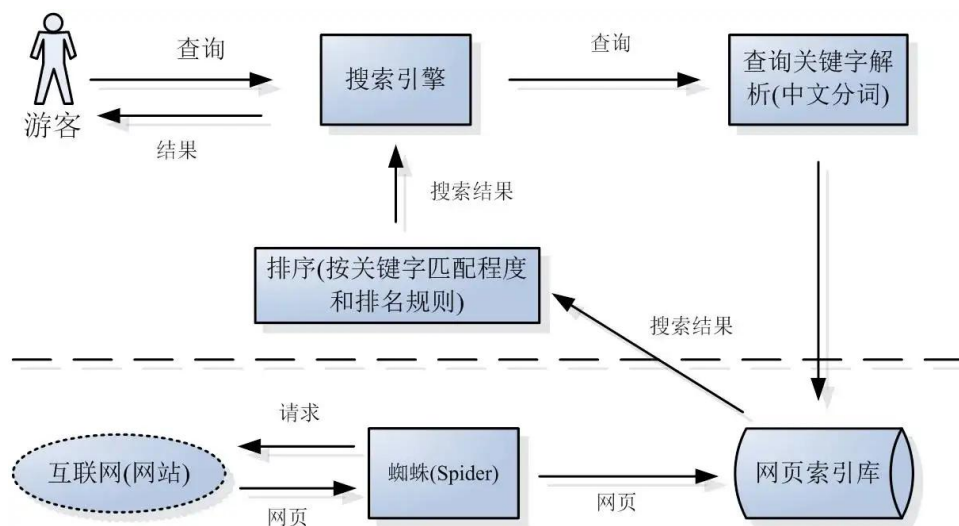
▶ 递归

- ▶ 利用栈实现子过程的逐层调用
- ▶ 递归有三个条件：可终止返回，调用自身，可分解为简单的相似过程

▶ 例子：四则运算，汉诺塔

4. 串

- ▶ 4.1 串类型的定义
- ▶ 4.2 串表示和实现
- ▶ 4.3 串的模式匹配算法





4.1 串的定义

- 串(String)，是由零个或多个字符组成的有限序列，也称字符串。一般记为：

$$s = 'a_1a_2a_3...a_n'$$

其中s称为串名，是用单引号括起来的字符序列

C语言中字符串是用双引号括起来的字符序列

a_i ($1 \leq i \leq n$)是字符，可以是字母、数字或其它符号

n是串s的长度。

- 零个字符的串称为空串null -> C语言NULL
- 串中任意个连续的字符组成的子序列称为该串的子串。包含子串的串相应地称为主串。通常称字符在序列中的序号为该字符在串中的位置。子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。



串的例子

- 称两个串是相等的，当且仅当这两个串的值相等。也就是说，只有当两个串的长度相等，并且各个对应位置的字符都相等时才相等。

a = 'WU',

b = 'HAN'

c = 'WUHAN'

d = 'WU HAN'

- 值得一提的是，串值必须用一对单引号括起来，但单引号本身不属于串，它的作用只是为了避免与变量名或数的常量混淆而已。



串的操作

基本操作:

StrAssign (&T, chars)

初始条件:chars 是字符串常量。

操作结果:生成一个其值等于 chars 的串 T。

StrCopy (&T, S)

初始条件:串 S 存在。

操作结果:由串 S 复制得串 T。

StrEmpty (S)

初始条件:串 S 存在。

操作结果:若 S 为空串,则返回 TRUE,否则返回 FALSE。

StrCompare (S, T)

初始条件:串 S 和 T 存在。

操作结果:若 $S > T$,则返回值 > 0 ;

若 $S = T$,则返回值 $= 0$;

若 $S < T$,则返回值 < 0 。

StrLength (S)

初始条件:串 S 存在。

操作结果:返回 S 的元素个数,称为串的长度。

ClearString (&S)

初始条件:串 S 存在。

操作结果:将 S 清为空串。

Concat (&T, S1, S2)

初始条件:串 S1 和 S2 存在。

操作结果:用 T 返回由 S1 和 S2 联接而成的新串。

SubString (&Sub, S, pos, len)

初始条件:串 S 存在, $1 \leq pos \leq \text{StrLength}(S)$

且 $0 \leq len \leq \text{StrLength}(S) - pos + 1$ 。

操作结果:用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。

Index (S, T, pos)

初始条件:串 S 和 T 存在,T 是非空串,

$1 \leq pos \leq \text{StrLength}(S)$ 。

操作结果:若主串 S 中存在和串 T 值相同的子串,则返回它的主串 S 中第 pos 个字符之后第一次出现的位置;否则函数值为 0。

Replace (&S, T, V)

初始条件:串 S,T 和 V 存在,T 是非空串。

操作结果:用 V 替换主串 S 中出现的所有与 T 相等的、不重叠的子串。

StrInsert (&S, pos, T)

初始条件:串 S 和 T 存在, $1 \leq pos \leq \text{StrLength}(S) + 1$ 。

操作结果:在串 S 的第 pos 个字符之前插入串 T。

StrDelete (&S, pos, len)

初始条件:串 S 存在, $1 \leq pos \leq \text{StrLength}(S) - len + 1$ 。

操作结果:从串 S 中删除第 pos 个字符起长度为 len 的子串。

DestroyString (&S)

初始条件:串 S 存在。

操作结果:串 S 被销毁。



串的基本操作

- ▶ 最小操作子集：StrAssign、StrCompare、StrLength、Concat、SubString。基于此，可以进行子串查找定位。

```
int Index(String S, String T, int pos) {  
    if(pos > 0) {  
        n = StrLength(S); m = StrLength(T); i = pos;  
        while(i <= n - m + 1) {  
            SubString(sub, S, i, m);  
            if(StrCompare(sub, T) != 0) ++i;  
            else return i;  
        } //while  
    } // if  
    return 0;  
} //Index
```



4. 2串的实现

► 定长顺序存储表示

```
#define MAXSTRLEN 255;
```

```
typedef unsigned char SSTRING[MAXSTRLEN+1];
```



字符数组的连接

```
Status Concat(SString &T, SString S1, SString S2) {  
    if(S1[0]+S2[0] <= MAXSTRLEN) {  
        T[1..S1[0]] = S1[1..S1[0]];  
        T[S1[0]+1..S1[0]+S2[0]] = S2[1..S2[0]];  
        T[0] = S1[0] + S2[0];  
    }  
    else if(S1[0] < MAXSTRLEN) {    // 截断  
        ...  
    }  
    ...  
}
```



串的堆分配存储表示

► 堆分配存储表示

```
Typedef struct {  
    char *ch;  
    int length;  
} Hstring;
```



连接操作

```
Status Concat(Hstring &T, HString S1, HString S2) {  
    if(T.ch) free(T.ch);  
    if(!(T.ch = (char *)malloc((S1.length+S2.length)*sizeof(char))))  
        exit(OVERFLOW);  
    T.ch[0..S1.length-1] = S1.ch[0..S1.length-1];  
    T.length = S1.length + S2.length;  
    T.ch[S1.length..T.length-1] = S2.ch[0..S2.length-1];  
    return OK;  
}
```



串的插入操作

```
Status StrInsert(HString &S, int pos, HString T) {  
    if(pos<1||pos>S.length+1) return ERROR;  
    if(T.length) {  
        if(!(S.ch = (char *)realloc(S.ch, S.length+T.length)*sizeof(char)))  
            exit(OVERFLOW);  
        for(i=S.length-1; i>=pos-1; --i)  
            S.ch[i+T.length] = S.ch[i];  
        S.ch[pos-1..T.length-2] = T.ch[0..T.length-1];  
        S.length += T.length;  
    }  
    return OK;  
}
```



串的块链存储表示

```
#define CHUNKSIZE 80
typedef struct Chunk {
    char ch[CHUNKSIZE];
    struct Chunk *next;
} Chunk;
typedef struct {
    Chunk *head, *tail;
    int curlen;
} LString;
```

$$\text{存储密度} = \frac{\text{串值所占的存储位}}{\text{实际分配的存储位}}$$



4.3 串的模式匹配算法

- ▶ 串的模式匹配：子串的定位操作。
- ▶ 子串称为模式串，主串称为目标。
- ▶ 定长顺序存储结构的匹配算法

```
int Index(SString S, SString T, int pos) {  
    i = pos; j = 1;  
    while(i <= S[0] && j <= T[0]) {  
        if(S[i] == T[j]) {++i; ++j;}  
        else {i = i - j + 2; j = 1;}  
    }  
    if(j > T[0]) return i - T[0];  
    else return 0;  
}
```

例： 目标S: Huazhong University
模式T: ver
匹配结果： 13



朴素的模式匹配 (B-F算法)

第1趟

S	a	b	b	a	b	a
T	a	b	a			

$i=3$
 $j=3$

第3趟

S	a	b	b	a	b	a
T			a	b	a	

$i=3$
 $j=1$

第2趟

S	a	b	b	a	b	a
T		a	b	a		

$i=2$
 $j=1$

第4趟

S	a	b	b	a	b	a
T				a	b	a

$i=6$
 $j=4$

计算复杂度:

最坏情况: T为m, S为n, 需要 $(n-m+1)*m$ 次比较, 大多 $m \ll n$, 故复杂度为 $O(n*m)$

低效原因: 每趟重新比较时, 目标串的检测指针要回溯



改进的模式匹配

- ▶ D.E.Knuth、J.H.Morris和V.R.Pratt同时提出来的，故称为KMP算法
- ▶ 核心思想：**消除**每趟失配后的**指针回溯**，即可提高模式匹配效率

指针回溯： $i=i-j+2, j=1$

第1趟

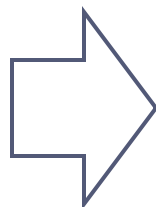
S	a	b	<u>b</u>	a	b	a
T	a	b	a			

$i=3$
 $j=3$

第2趟

S	a	<u>b</u>	<u>b</u>	a	b	a
T		a	b	a		

$i=2$
 $j=1$



指针回溯： $i=3, j=1$

第3趟

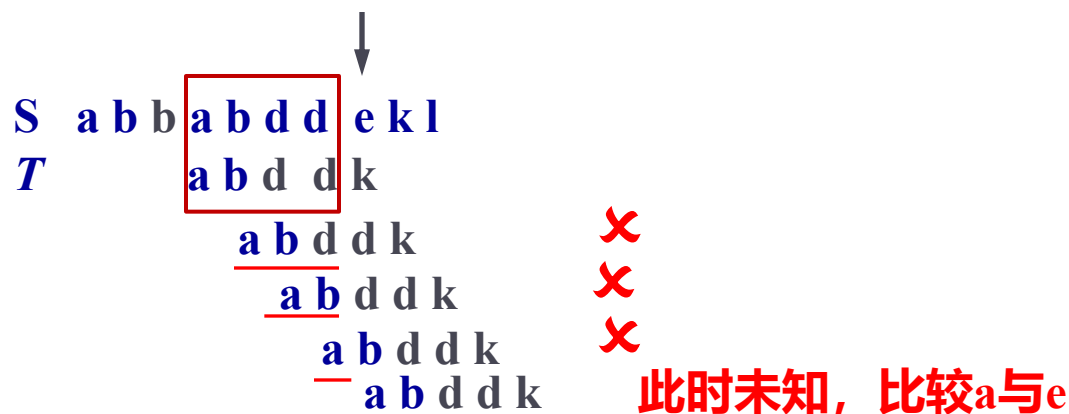
S	a	b	<u>b</u>	a	b	a
T			a	b	a	

$i=3$
 $j=1$



带来的新问题

- ▶ 若i不回溯，j应该退到何处？
- ▶ 给定模式串，用next[j]表示新的位置，如何求？
- ▶ next[j]与主串S无关，只与模式串T有关？为什么？





运用KMP算法的匹配过程

第1趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$

× $j=2 \Rightarrow \text{next}(2) = 1$, 下次 p_1

第2趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$

× $j=1 \Rightarrow$ 下次 p_1 , 目标指针进 1

第3趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $a b a a b c a c$

× $j=6 \Rightarrow \text{next}(6) = 3$,
下次 p_3

第4趟 目标 $a c a b a a b a a b c a c a a b c$
模式 $(a b) a a b c a c \quad \checkmark$



改进原理

$$\begin{array}{ccccccccccc}
 S & & s_1 & \cdots & s_{i-j} & s_{i-j+1} & s_{i-j+2} & s_{i-j+3} & \cdots & s_{i-1} & s_i & s_{i+1} & \cdots & s_{n-1} \\
 & & & & & \parallel & \parallel & \parallel & \parallel & & \times & & & \\
 T & & & & & p_1 & p_2 & p_3 & \cdots & p_{j-1} & p_j & \cdots & p_m &
 \end{array}$$

则有 $s_{i-j+1} s_{i-j+2} s_{i-j+3} \cdots s_{i-1} = p_1 p_2 \cdots p_{j-1} \quad (1)$

按照依次回溯法：下一趟，为使模式 T 与目标 S 匹配，必须满足

$$p_1 p_2 \cdots p_{j-1} \cdots p_m = s_{i-j+2} s_{i-j+3} \cdots s_{i-j+2+m}$$

如果 $p_1 \cdots p_{j-2} \neq p_2 \cdots p_{j-1} \quad (2)$

则立刻可以断定

$p_1 \cdots p_{j-2} \neq s_{i-j+2} s_{i-j+3} \cdots s_{i-1}$ **下一趟必不匹配**

改进原理-续

$$\begin{array}{ccccccccccc}
 S & & s_1 & \dots & s_{i-j} & s_{i-j+1} & s_{i-j+2} & s_{i-j+3} & \dots & s_{i-1} & s_i & s_{i+1} & \dots & s_{n-1} \\
 & & & & & \parallel & \parallel & \parallel & \parallel & & \times & & & \\
 T & & p_1 & p_2 & p_3 & \dots & p_{j-1} & p_j & \dots & p_m & & & &
 \end{array}$$

同样的道理： $p_1 \dots p_{j-3} \neq p_3 p_4 \dots p_{j-1}$ 下一趟必不匹配

直到某个k, $p_1 \dots p_k \neq p_{j-k} p_{j-k} \dots p_{j-1}$

但, $p_1 \dots p_{k-1} == p_{j-k+1} p_{j-k+1} \dots p_{j-1}$

故有： $p_1 \dots p_{k-1} == s_{i-k+1} s_{i-k+1} \dots s_{i-1}$

$p_{j-k+1} p_{j-k+1} \dots p_{j-1}$

下一趟可以直接用 p_k 与 s_i 继续比较。



$p_1 \dots p_{k-1} == p_{j-k+1} p_{j-k+1} \dots p_{j-1}$

仅和模式串有关，如何确定k?



K的确定

- ▶ 当第j个字符失配时，不妨令 $k = \text{next}[j]$ 表示
- ▶ 它仅依赖于模式T本身前j个字符的构成，与目标无关. 设模式 $T = p_1 \dots p_{m-1} p_m$

$$\text{next}(j) = \begin{cases} 0, & j=1 & \text{子串保持不变, 主串下移1位} & (1) \\ k, & 1 < k < j \text{ 且使得 } p_1 \dots p_{k-1} = p_{j-k+1} p_{j-k} \dots p_{j-1} \text{ 的最大整数} & (2) \\ 1, & \text{其它情况} & (3) \end{cases}$$

j	1	2	3	4	5	6	7	8
T	a	b	a	a	b	c	a	c
next(j)	0	1	1	2	2	3	1	2



KMP算法实现代码

```
int Index_KMP(SString S, SString T, int pos) {  
    // 利用模式串 T 的 next 函数求 T 在主串 S 中第 pos 个字符之后的位置的  
    // KMP 算法。其中, T 非空,  $1 \leq \text{pos} \leq \text{StrLength}(S)$ 。  
    i = pos;    j = 1;  
    while (i <= S[0] && j <= T[0]) {  
        if (j == 0 || S[i] == T[j]) { ++i; ++j; }           // 继续比较后继字符  
        else j = next[j];                                   // 模式串向右移动  
    }  
    if (j > T[0]) return i - T[0];                          // 匹配成功  
    else return 0;  
} // Index_KMP
```

算法 4.6

$p_1 \ p_2 \ p_3 \ \cdots \ p_{j-k+1} \ \cdots \ p_{j-1} \ p_j \ p_{j+1} \ \cdots p_m$

$p_1 \ \cdots \ p_{k-1} \ \cdots \ p_x$
 p_k



求next[j+1]

$$\text{next}(j) = \begin{cases} 0, & j=1 \quad \text{子串保持不变, 主串下移1位} & (1) \\ k, & 1 < k < j \text{ 且使得 } p_1 \cdots p_{k-1} = p_{j-k+1} p_{j-k} \cdots p_{j-1} \text{ 的最大整数} & (2) \\ 1, & \text{其它情况} & (3) \end{cases}$$

若已知next[j]=k, 则 $p_1 \cdots p_{k-1} = p_{j-k+1} \cdots p_{j-1}$
 且k为满足上式最大的值, 因此j处失配时
 只需比较 p_k 与 p_j 的值:

- (1) 若相等, 显然与 p_{j+1} 比较的应为 p_{k+1}
- (2) 若不等, 相当于字符串在k处失配, 下一步转到next[k]与 p_j 再次比较

▶ 递推思想: 即已知next[l]=0, ..., next[j]=k,
 求next[j+1]。

① 判断模式串的 p_j 是否等于 p_k 。

② 若两者相等, 有next[j+1]=k+1;

否则由于模式串在 p_k 处失配, 应将模式串的next[k]
 与 p_j 再做比较 (相当于子串右滑), 即取 $k=\text{next}[k]$, 转
 ①。

Take-home Message



- 串的基本概念和基本操作算法
- 串的简单模式匹配算法和KMP算法
- KMP算法利用了模式串内在的相似性简化指针移动